**NAVSAHYADRI GROUP OF INSTITUES (NGI), PUNE**
(Affiliated to Savitribai Phule Pune University,Approved by AICTE
& Accredited by NAAC )

A

PROJECT

REPORT ON

TITLE OF PROJECT

# Smart Agriculture Platform

By

**MS.PRANJALI JALINDAR NIKAM**

MCA – I SEM-I
(2025-2026)

**IN PARTIAL FULFILLMENT OF MASTER OF COMPUTER APPLICATION (MCA)**

**SUBMITTED TO**

**SAVITRIBAI PHULE PUNE UNIVERSITY (SPPU)**

NAVSAHYADRI GROPU OF INSTITUTIONS
FACULTY OF MANAGEMENT
Approved by AICTE New Delhi
Affiliated to Savitribai Phule Pune University
Address: - Sr. No. 69,70,71 NaigaonNasarapurPune, Maharashtra 412213.
Website: https://www.navsahyadri.edu.in/E- mail- director@navsahyadri.edu.in

| Mr. P N Suke | Prof. Sagar Suke | Dr. Tanaji Dabade |
|---|---|---|
| President | Group Director | Director |


# CERTIFICATE

Date:-

This is to certify that Mr. / Ms. _____, has successfully / partially completed his/her project work entitled "__" in partial fulfillment of MCA-I SEM-I Mini Project for the year 2025-2026. He / She have worked under our guidance and direction.

**Prof.Shubhangi Pharande**
**Project Guide**

**Dr. Balasaheb Bhamangol**
**Professor & HoD - MCA**

**Examiner 1**

**Examiner 2**

**Date :**

**Place :**

# ACKNOWLEDGMENT

I am grateful to the NAVSAHYADRI GROUP OF INSTITUTES (NGI), Pune for providing me the opportunity to carry out this project successfully. I would like to express my deepest gratitude to my project guide **Prof. Shubhangi Pharande** for her continuous guidance, valuable suggestions and motivation throughout the development of this project. His support helped me understand concepts clearly and complete my project work on time.

I also wish to thank all the faculty members of the MCA Department for their constant encouragement and for providing academic resources whenever required. Their teaching and support were extremely helpful in improving my knowledge and skills.

I extend my special thanks to my classmates, friends and family for their support, cooperation and motivation during the entire project duration.
Their positive encouragement helped me stay focused and complete the work with confidence.

Lastly, I would like to thank everyone who directly or indirectly contributed to the completion of my project.

**Thank You,**
**Ms.Pranjali Jalindar Nikam**

# INDEX


| Chapter | Page number |
|---|---|
| **CHAPTER 1 : INTRODUCTION** | |
| 1.1 Existing System | |
| 1.2 Need for System | |
| 1.3 Operating Environment Hardware and Software | |
| **CHAPTER 2 : PROPOSED SYSTEM** | |
| 2.1 Proposed System (Introduction of system) | |
| 2.2 Module specifications (Scope) | |
| 2.3 Objectives of System | |
| **CHAPTER 3 : ANALYSIS & DESIGN** | |
| 3.1 Use Case Diagrams | |
| 3.2 Activity Diagram | |
| 3.3 Class Diagram | |
| 3.3 Module Hierarchy Diagram | |
| 3.4 Table specifications(Database design) | |
| 3.5 Data dictionary | |
| **CHAPTER 4 : USER MANUAL** | |
| 4.1 User Interface Screens (Input) | |
| 4.2 Output Screens with data | |
| 4.3 Data Reports | |
| 4.4 Sample program code | |
| 4.5 Limitations and Bibliography | |

# CHAPTER 1: INTRODUCTION

# 1.1 Existing System :

- 1.**Manual and Fragmented Agricultural Management**
  - Farm-related data such as crop prices, farm conditions, and predictions are handled separately.
  - There is no single integrated platform for farmers and administrators.
- **No Real-Time Automated Market Pricing**
  - Market prices are not fetched from real markets.
  - Prices are manually maintained or randomly generated without real-time market linkage.
- **Limited Monitoring of Farm Conditions**
  - Farm data like temperature, humidity, and soil moisture is stored, but:
    - Data is saved record-by-record
    - No advanced analytics or alert system exists in the existing setup.
- **Traditional Data Storage Approach**
  - Data is stored directly in MySQL tables.
  - No use of data warehousing or big data processing.
- **Separate Tools Without Central Analysis**
  - Yield prediction, disease detection, rainfall prediction, etc. exist as individual tools.
  - Outputs are logged, but cross-tool analysis is not automated.
- **Admin-Dependent Data Management**
  - Editing and deleting of records must be done manually by an admin.
  - No self-service dashboards for automated insights.
- **Basic Security Mechanism**
  - Role-based access is present (admin vs user).
  - No advanced security features like token-based APIs or encryption.
- **Limited Automation**
  - Only one automated task exists:
    - Hourly price update using APScheduler
  - Other operations rely on manual triggers or API calls.
- **User Interaction Through Web Interface Only**
  - Farmers and users interact mainly via web pages and basic APIs.
  - No mobile app or push notification system.
- **Lack of Predictive Decision Support**
  - Predictions are displayed but not linked to real-time decision-making.
  - No automated recommendations based on live sensor data.

# 1.2 Need for System :

1. **Centralized Agricultural Platform**
    - There is a need to combine market prices, farm sensor data, and prediction tools into a **single integrated system**.
    - Your app.py addresses this by managing all features under one Flask application.
2. **Automation of Repetitive Tasks**
    - Manual price updates are inefficient.
    - The system is needed to automatically update crop prices at regular intervals using a scheduler.
3. **Real-Time Farm Monitoring**
    - Farmers need real-time access to temperature, humidity, and soil moisture data.
    - The system enables continuous data collection through APIs.
4. **Data-Driven Decision Making**
    - Farmers require insights based on stored farm data rather than assumptions.
    - The system stores historical sensor data for analysis and future predictions.
5. **Administrative Control and Transparency**
    - A structured admin panel is required to:
        - Monitor tool usage
        - Edit or delete incorrect records
        - Maintain system data integrity
6. **Scalable Data Management**
    - With increasing users and tools, manual data handling becomes impractical.
    - The system organizes data into multiple MySQL tables for scalability.
7. **Monitoring of AI Tool Usage**
    - To understand how prediction tools are being used, usage logging is necessary.
    - The system records input and output of each tool for auditing and analysis.
8. **Security and Role-Based Access**
    - Not all users should have access to sensitive data.
    - The system enforces admin-only access for management features.
9. **Improved Accuracy and Reliability**
    - Storing time-stamped data ensures accurate tracking of prices and farm conditions.
    - Reduces dependency on manual record keeping.

10. **Support for Future Enhancements**
    - The system is needed as a foundation to integrate:
        - Real market APIs
        - Advanced analytics
        - Mobile applications
    - Your app.py structure supports future expansion

# Operating Environment Hardware and Software:

## Hardware Requirements

1. **Server / Hosting Machine**
    - A system capable of running a Flask web server.
    - Can be a local machine, cloud server, or VPS.
2. **Processor**
    - Minimum **Intel i3 / AMD equivalent**
    - Recommended **multi-core processor** for handling multiple API requests.
3. **RAM**
    - Minimum **4 GB RAM**
    - Recommended **8 GB RAM** for smooth scheduler and database operations.
4. **Storage**
    - Minimum **20 GB free disk space**
    - Required for:
        - MySQL database
        - Uploaded images (static/uploads)
        - Application logs
5. **Network Connectivity**
    - Stable internet connection
    - Required for:
        - API communication
        - Admin dashboard access
        - Future integrations

## Software Requirements

1. **Operating System**
    - Windows / Linux / macOS
    - Linux preferred for deployment.
2. **Programming Language**
    - **Python 3.x**
    - Core language used in app.py.
3. **Web Framework**
    - **Flask**
    - Used to create routes, APIs, templates, and admin panel.
4. **Database**
    - **MySQL**
    - Used via flask_mysqldb for storing:
        - Users
        - Prices
        - Farm data
        - Tool usage logs
        - Admin records
5. **Database Connector**
    - flask-mysqldb
    - Enables MySQL connection within Flask.
6. **Scheduler**
    - **APScheduler**
    - Used for automated hourly crop price updates.
7. **Frontend Technologies**
    - **HTML, CSS, JavaScript**
    - Jinja2 templating used with Flask.
8. **Libraries & Modules**
    - datetime – timestamp handling
    - random – price simulation
    - os – file and directory handling
    - json – tool usage logging

9. **Web Browser**
    - Any modern browser (Chrome, Edge, Firefox)
    - Used to access user and admin dashboards.

10. **Optional / Future Software**
    - Cloud deployment tools (AWS, Azure, etc.)

      Mobile app integration (API-based)

    - Machine learning models for predictions

# CHAPTER 2: PROPOSED SYSTEM

## 2.1 Proposed System (Introduction of System):

1. **Integrated Smart Agriculture Platform**
   - The proposed system is a **web-based smart agriculture management system** developed using Flask.
   - It integrates farm monitoring, market price updates, AI-based prediction tools, and administrative control into one platform.
2. **Automated Live Market Price Management**
   - The system automatically generates and updates crop prices at regular intervals using a background scheduler.
   - Prices are stored with timestamps and displayed to users in real time.
3. **IoT-Based Farm Data Collection**
   - The system provides APIs to collect farm sensor data such as:
     - Temperature
     - Humidity
     - Soil moisture
   - Data is stored continuously for monitoring and analysis.
4. **Multiple AI-Assisted Agricultural Tools**
   - The system supports various agricultural decision-support tools, including:
     - Yield prediction
     - Disease detection
     - Rainfall prediction
     - Seed quality analysis
     - Climate advisory
     - Harvest time prediction
   - Each tool's usage and output are logged in the database.
5. **Role-Based User and Admin Access**
   - Normal users can access farming tools and insights.
   - Admin users have exclusive access to dashboards for:
     - Viewing
     - Editing
     - Deleting records
6. **Centralized Data Storage**
   - All system data is stored in a structured MySQL database.
   - Enables reliable data retrieval, management, and auditing.
7. **Real-Time APIs and Web Interface**
   - REST APIs provide real-time data access for frontend or external devices.
   - Web interface allows easy interaction for farmers and administrators.

8. **Administrative Monitoring and Control**
    - The admin panel allows monitoring of:
        - Tool usage logs
        - User activities
        - Market prices
        - Farm sensor records
    - Ensures data accuracy and system transparency.

9. **Scalable and Extendable Architecture**
    - The system architecture supports future enhancements such as:
        - Real market data integration
        - Advanced analytics
        - Mobile application support
10. **Efficient and Reliable Decision Support**
    - By combining sensor data, automated pricing, and predictive tools, the system assists farmers in making informed agricultural decisions.

## 2.2 Module specifications (Scope):

### 1. User Management Module

- Handles registration, login, and role-based access.
- Stores user details such as name, email, phone, address, role, and profile photo.
- Allows admin to:
  - View all users
  - Edit user details
  - Delete users
- Ensures secure access using session-based authentication.

### 2. Live Market Price Module

- Automatically updates crop prices using a background scheduler.
- Stores crop name, price, and timestamp in the database.
- Displays daily prices to users on the home page.
- Provides APIs for fetching live prices.
- Admin can:
  - View price history
  - Edit prices
  - Delete prices

### 3. Farm Monitoring (IoT Data) Module

- Collects farm sensor data via REST APIs.
- Stores temperature, humidity, and soil moisture values.
- Maintains historical farm data with timestamps.
- Admin can:
  - View all sensor records
  - Edit or delete individual entries
- Supports real-time farm condition monitoring.

### 4. Yield Prediction Module

- Stores yield prediction inputs and results.
- Tracks user email, crop details, and predicted yield.
- Admin can view, edit, and delete prediction records.
- Helps farmers estimate crop yield.

### 5. Disease Detection Module

- Supports image-based disease detection.
- Stores uploaded images, predicted disease, and remedy.
- Provides paginated admin view for large datasets.
- Admin can edit and delete disease detection records.

### 6. Seed Quality Analysis Module

- Analyzes seed quality based on parameters like moisture and purity.
- Stores predicted viability and recommendations.
- Admin can manage seed quality records.

### 7. Rainfall Prediction Module

- Predicts rainfall based on user input and location.
- Stores predicted rainfall values and advisory messages.
- Admin can edit or delete rainfall prediction records.

### 8. Climate Advisory Modul

- Generates climate-based advisories using temperature, humidity, and rainfall.
- Stores advisory messages for reference.
- Admin can manage climate advisory data.

### 9. Mixed Crop Planning Module

- Suggests compatible crops with a main crop.
- Stores crop combinations for future reference.
- Admin can edit and delete mixed crop records.

### 10. Water Requirement Forecast Module

- Estimates crop water needs and irrigation schedules.
- Stores advice and next irrigation date.
- Admin can manage water forecast data.

### 11. Market Demand Forecast Module

- Predicts crop demand based on season and region.
- Stores demand levels and best crop suggestions.
- Admin can edit and delete market forecast records.

### 12. Harvest Time Prediction Module

- Predicts harvest date based on sowing date.
- Calculates crop duration and provides advice.
- Admin can edit and delete harvest prediction records.

### 13. Chatbot Interaction Module

- Records chatbot conversations.
- Stores user choices and bot replies.
- Admin can monitor, edit, and delete chatbot logs.

### 14. E-commerce Tools Module

- Tracks usage of e-commerce platforms.
- Stores platform name and user actions.
- Admin can manage e-commerce tool logs.

### 15. Contact Management Module

- Stores user contact messages.
- Admin can view, edit, and delete messages.
- Ensures communication between users and administrators.

**Overall Scope Summary**

- The system covers **end-to-end smart farming needs**:
  - Monitoring
  - Prediction
  - Advisory
  - Market insights
  - Administration
- Provides a scalable and centralized agricultural decision-support platform.

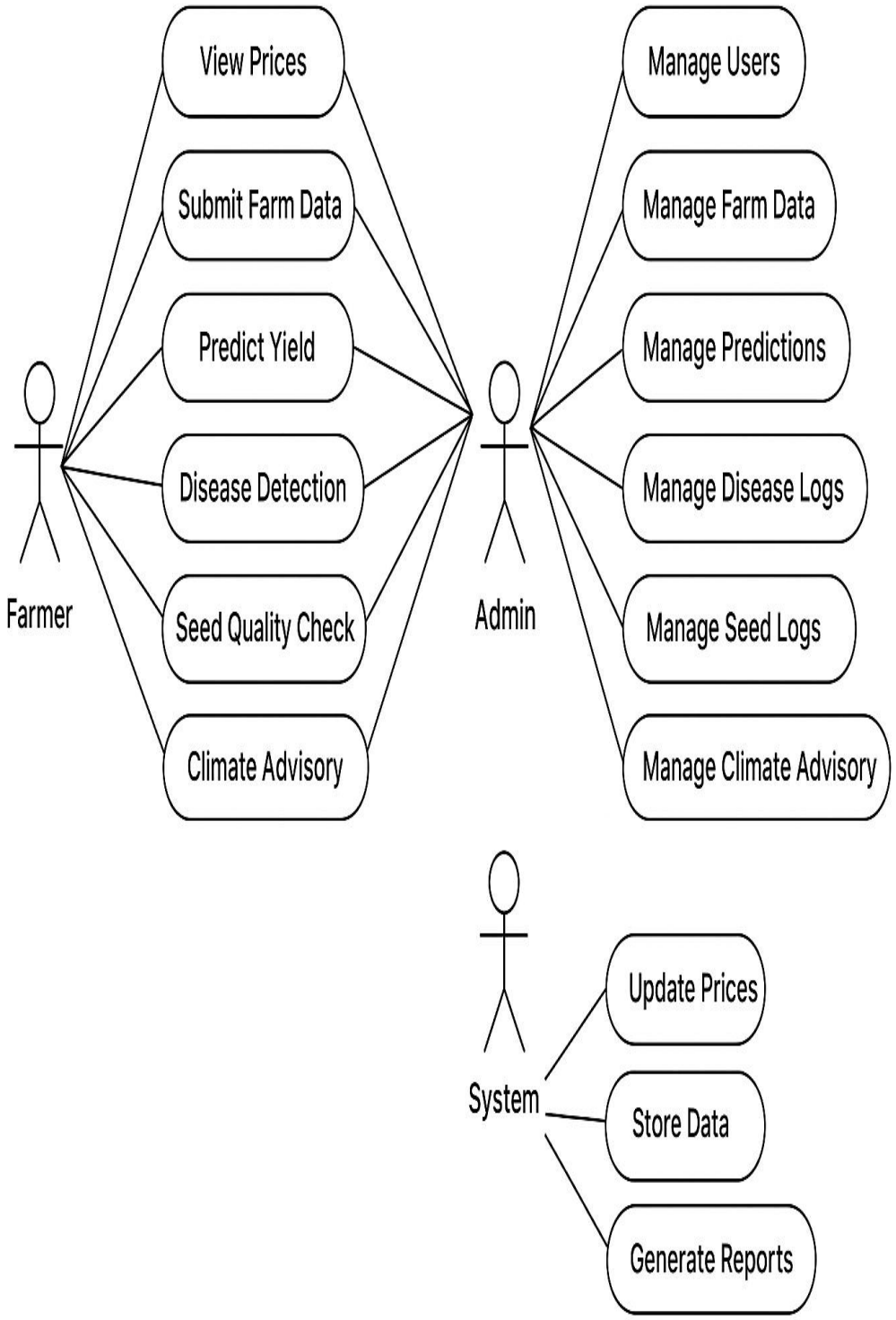
## 2.3 Objectives of System:

1. **Real-time Farm Monitoring**
    - Collect and store sensor data like temperature, humidity, and soil moisture.
    - Enable farmers to monitor field conditions remotely and make timely decisions.
2. **Live Market Price Tracking**
    - Provide daily updates of crop prices.
    - Help farmers make informed decisions about selling produce at the best market rates.
3. **Crop Yield Prediction**
    - Estimate potential crop yield based on environmental and input data.
    - Assist farmers in planning resources and marketing strategies.
4. **Disease Detection and Management**
    - Detect crop diseases from images uploaded by farmers.
    - Provide remedies and preventive measures to minimize crop losses.
5. **Seed Quality Analysis**
    - Assess seed quality using parameters like moisture and purity.
    - Recommend actions for improving seed viability.
6. **Rainfall and Climate Advisory**
    - Predict rainfall and provide climate-related advisory messages.
    - Help farmers plan irrigation, sowing, and harvesting schedules.
7. **Mixed Crop and Water Requirement Planning**
    - Suggest compatible crops and optimize water usage.
    - Promote sustainable farming practices and resource efficiency.
8. **Market Demand and Harvest Prediction**
    - Forecast market demand for different crops.
    - Predict harvest dates and crop durations for better planning.
9. **Administrative Control**
    - Allow admins to manage users, farm data, predictions, and logs.
    - Maintain system integrity, data accuracy, and user accountability.
10. **Enhanced Decision Support for Farmers**
    - Integrate all modules into a single platform for centralized farm management.
    - Enable data-driven decisions to improve productivity, reduce losses, and maximize profits.

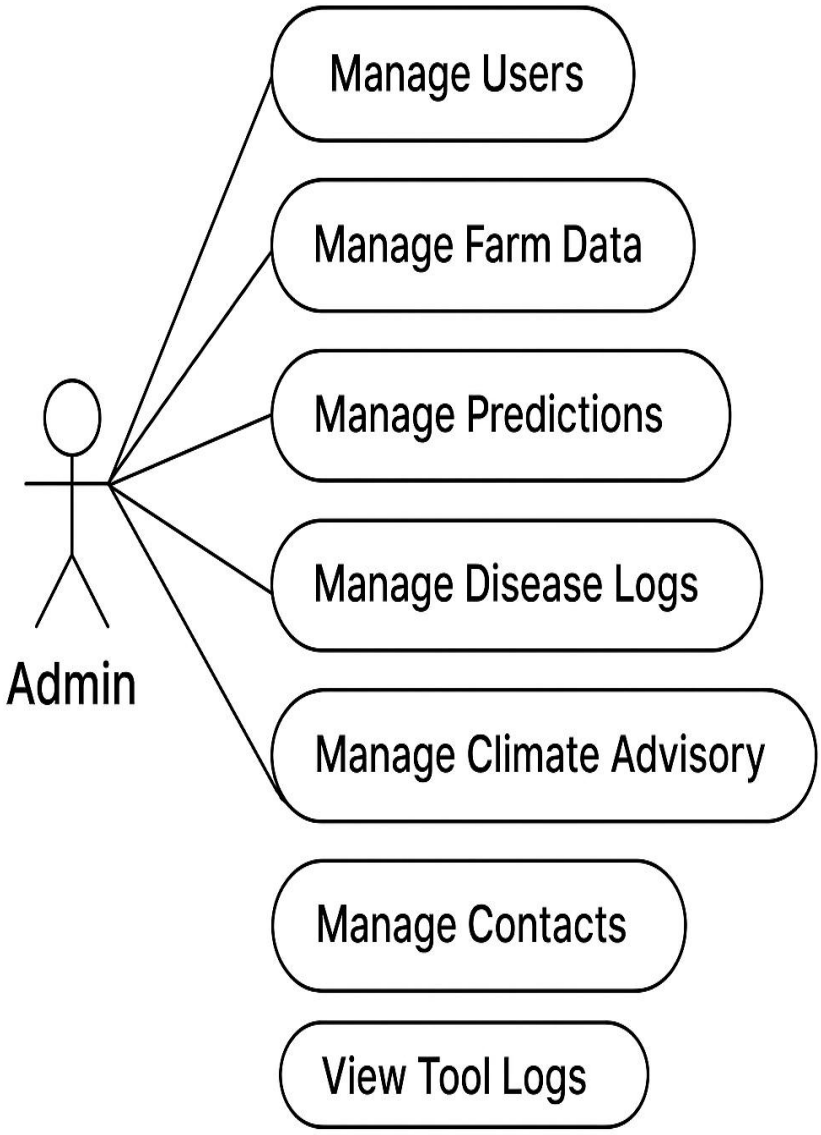
# CHAPTER 3: ANALYSIS & DESIGN

## 3.1 Use Case Diagrams :

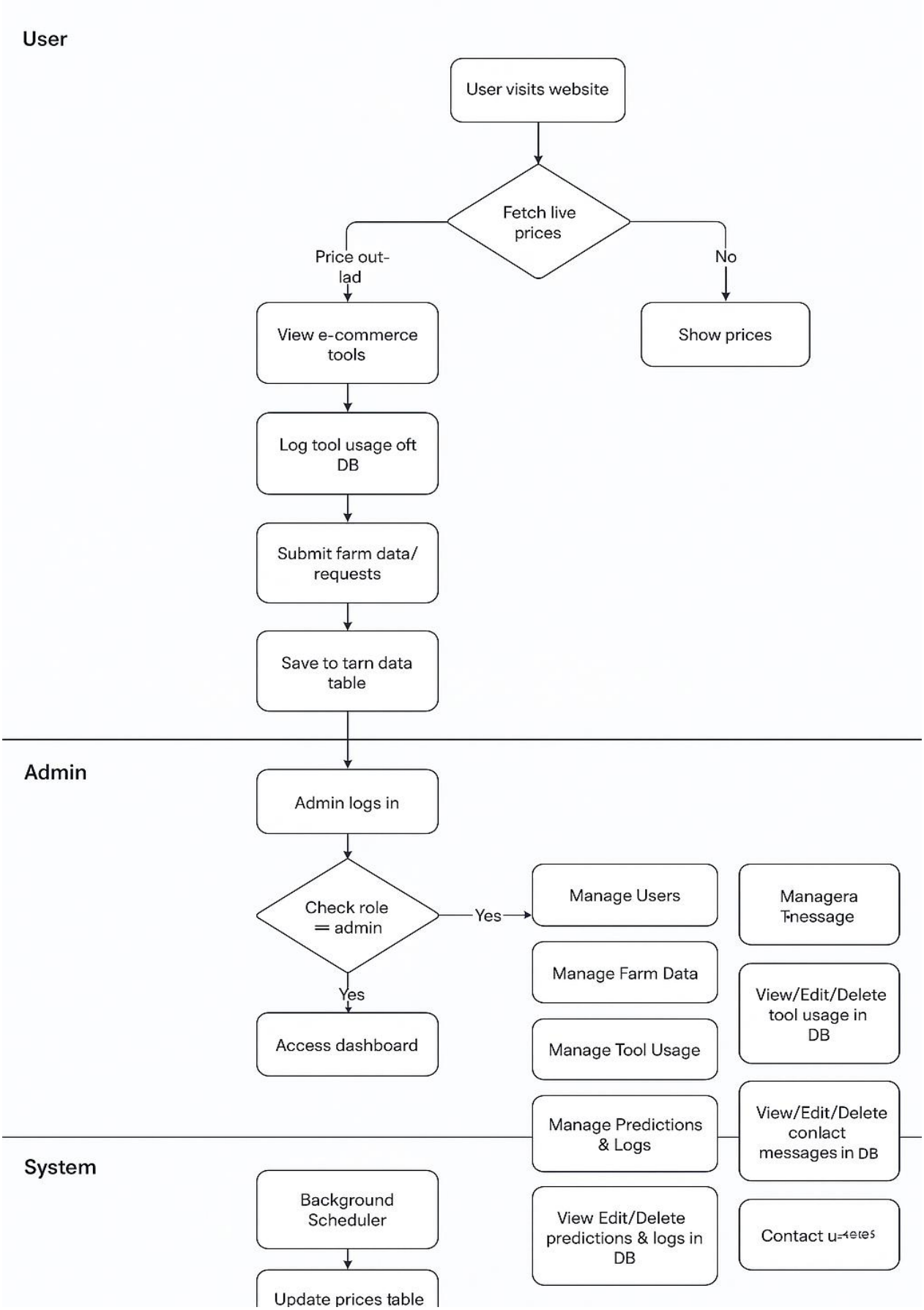
### I) User class :



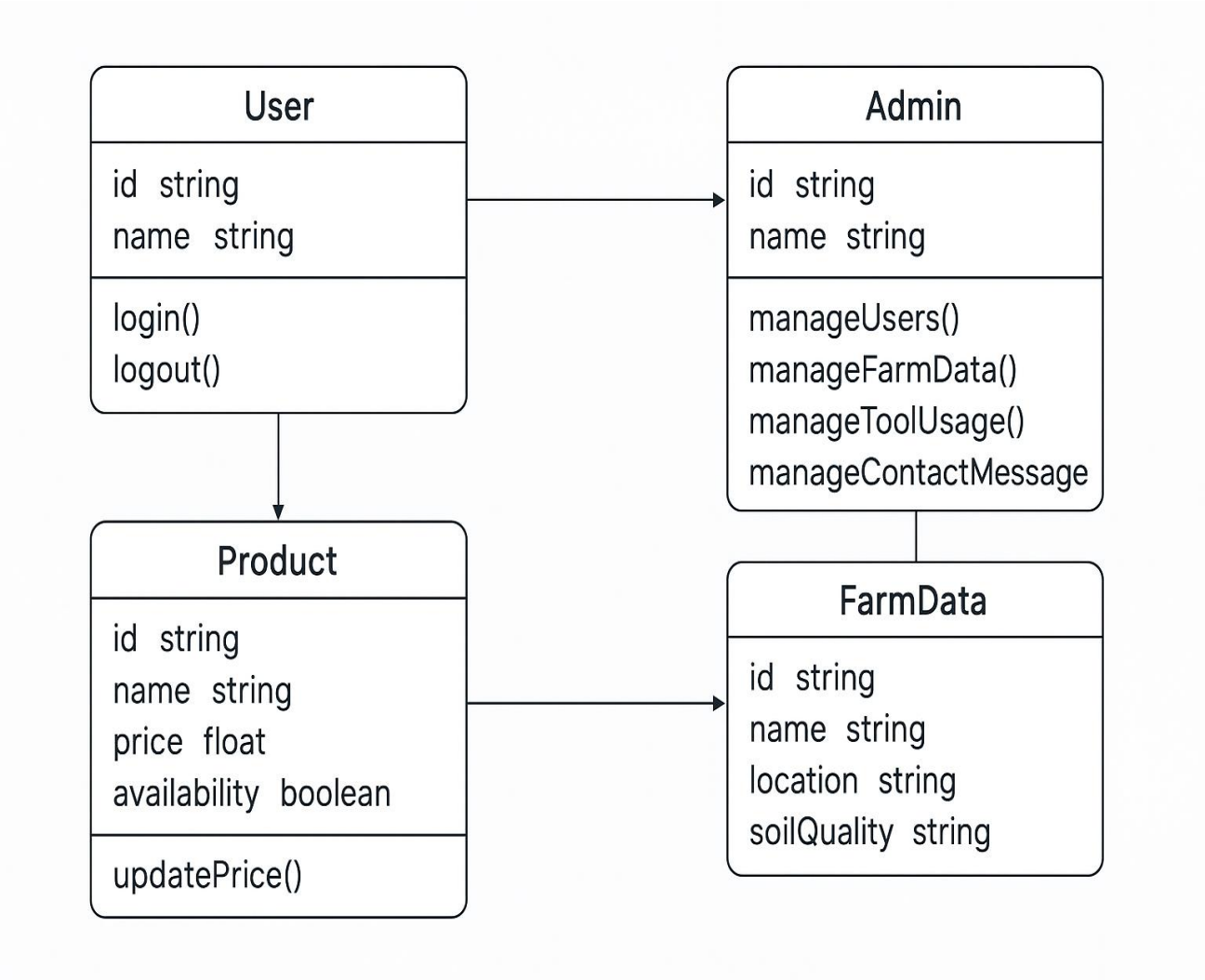
**II)Admin class :**



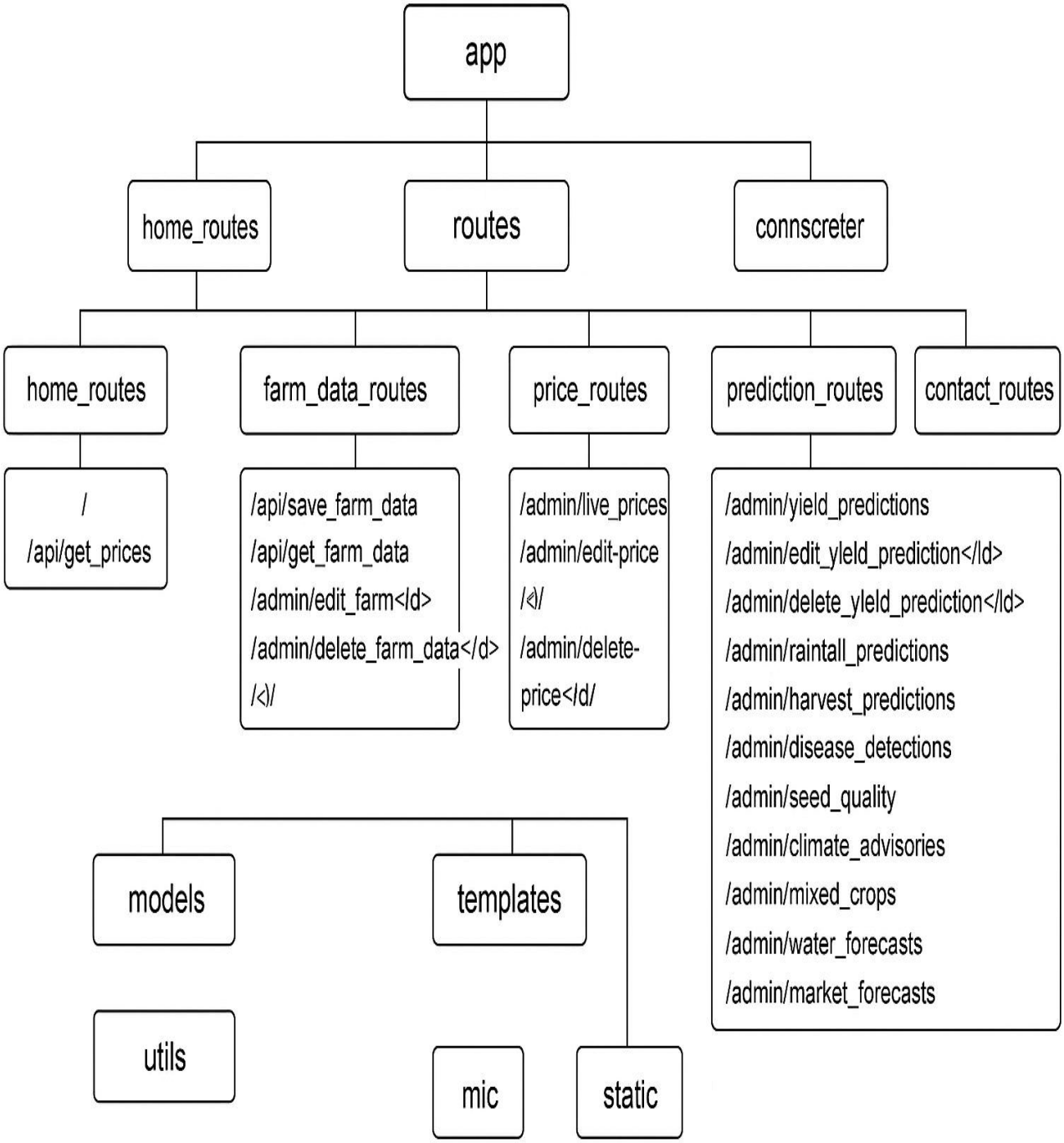
## 3.2 ActivityDiagram:



## 3.3 Class Diagram :



## 3.4 Module Hierarchy Diagram :



## 3.5 Table specifications (Database design):

### 1. User Table (customer details):

Stores login and registration details of users.

| Field Name | Data Type | Key Type | Size | Description |
|---|---|---|---|---|
| id | INT | PRIMARY | 11 | Unique user ID |
| fullname | VARCHAR | UNIQUE | 255 | Full name of |
| email | VARCHAR | | 255 | User email |
| password | VARCHAR | | 255 | Hashed |
| phone | VARCHAR | | 20 | User phone |
| address | VARCHAR | | 255 | User address |
| profile_photo | VARCHAR | | 255 | Path to profile |
| role | DECIMAL | | 20 | Role of user |
| latitude | DECIMAL | | 10,7 | Latitude locat- |

### 2. Farm Data Table (Stores farmer/farm-specific data)

| Field Name | Data Type | Key Type | Size | Description |
|---|---|---|---|---|
| id | INT | PRIMARY | 11 | Unique identi-fier for each record |
| temperature | DECIMAL(5, 2) | – | 5,2 | Temperature recorded in °C |
| humidity | INT | – | 3 | Humidity |
| soil_moisture | INT | – | 3 | Soil moisture |
| date_time | DATETIME | – | – | Date and time of the record |

**Prices Table (Stores market prices for crops)**

## prices

| Field Name | Data Type | Key Type | Size | Description |
|---|---|---|---|---|
| id | INT | Primary Key | 11 | Unique identifier for each record (Auto Increment) |
| crop_name | VARCHAR | | 50 | Name of the crop (Tomato, Onion, Wheat, etc.) |
| price | VARCHAR | | 20 | Crop price including currency and unit (e.g. |
| date_time | DATETIME | | | Date and time |

**Recommendations Table (Stores advisory messages or recommendations for Farmers.)**

| Field Name | Data Type | | Size | Description |
|---|---|---|---|---|
| id | INT | Primary Key | 11 | Unique identifier for each recommendation record (Auto Increment) |
| soil_type | VARCHAR | | 50 | Type of soil (Loamy, Black, Arid, Peat, Red, etc.) |
| nitrogen | INT | | 3 | Nitrogen level in the soil |
| phosphorus | INT | | 3 | Phosphorus level in the soil |
| potassium | INT | | 3 | Potassium level in the soil |
| ph | DECIMAL | | 4,2 | Soil pH value |
| temperature | INT | | 3 | Recommended temperature |
| humidity | INT | | 4 | Recommended rainfall (( %) |

## 5.Admin Table(Optional )

Used if admin login require for system

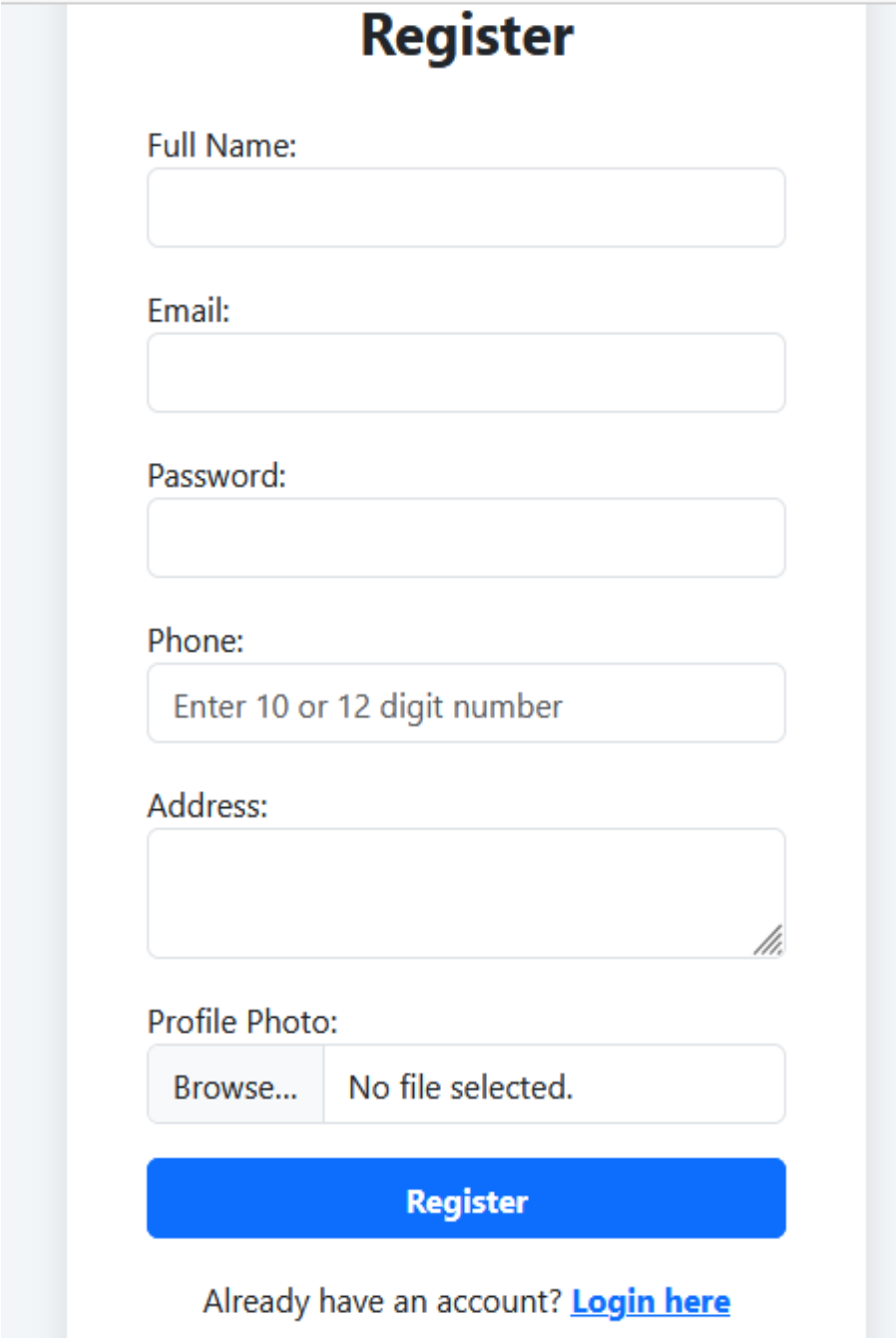
# Table: ADMIN

| Field Name | Data Type | Key Type | Size | Description |
|---|---|---|---|---|
| admin_id | INT | Primary Key | 11 | Unique admin ID |
| username | VARCHAR | — | 100 | Admin username |
| password | VARCHAR | — | 255 | Encrypted password |

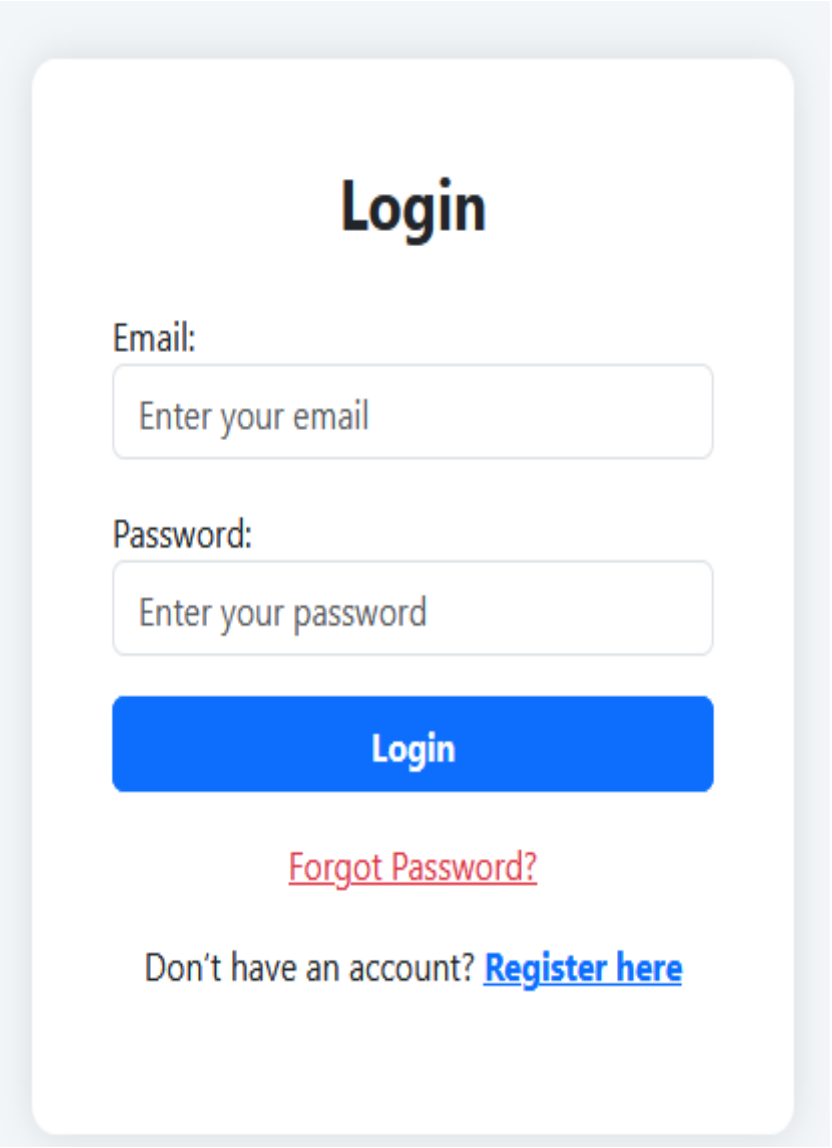
# CHAPTER 4 : USER MANUAL

## 2.1 User Interface Screens (Input) :

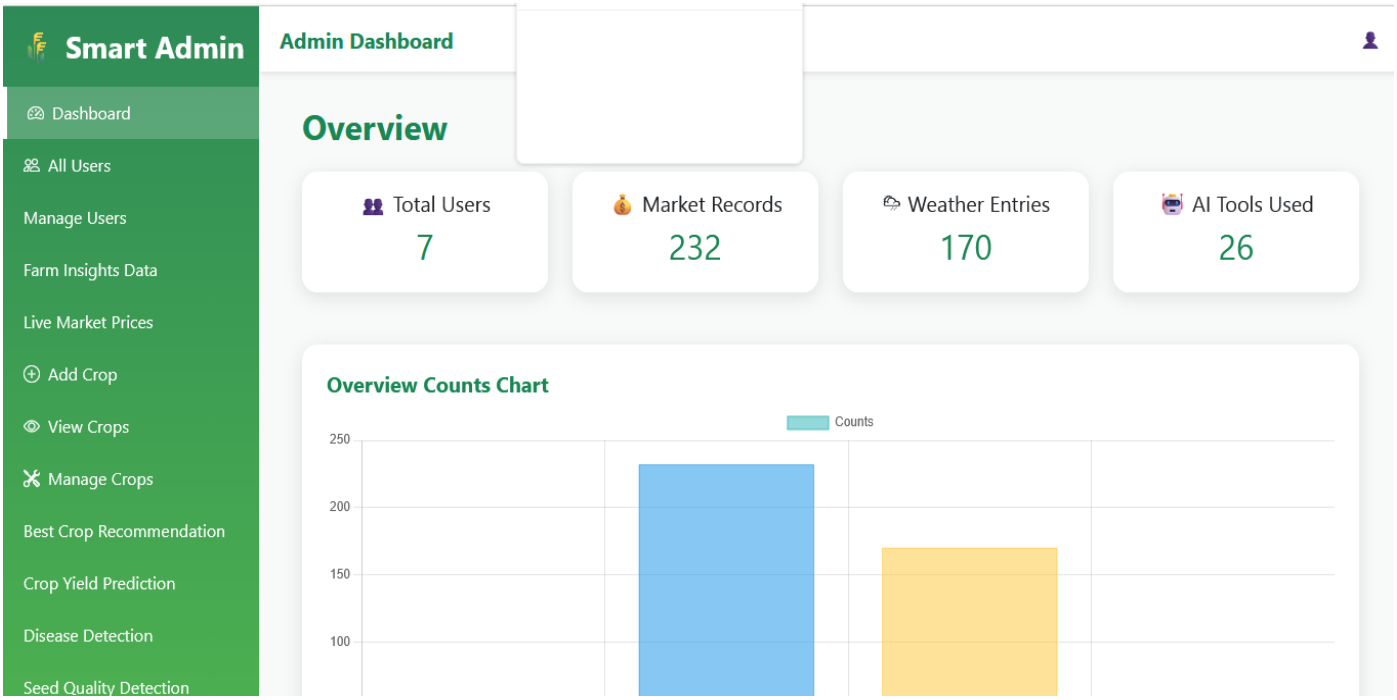
### I) Registration Page :



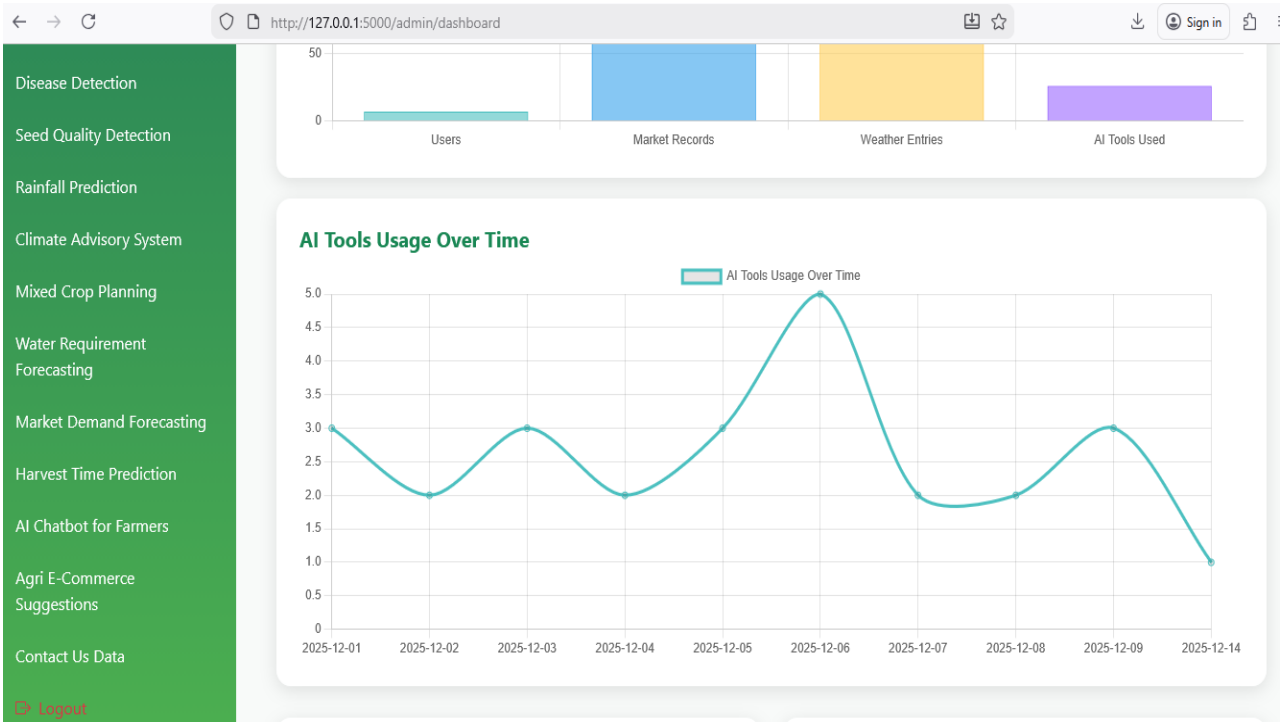
## II)Login Page



# Dash-board 1:

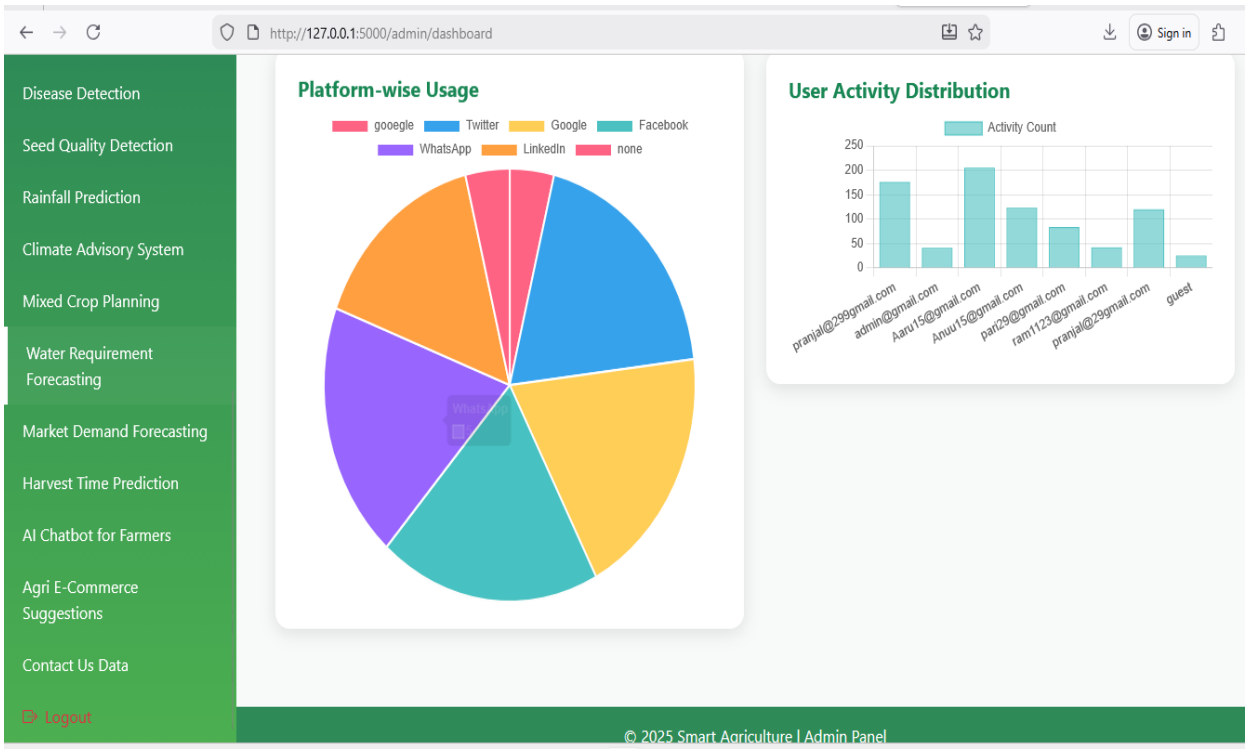



# Dash-board 2

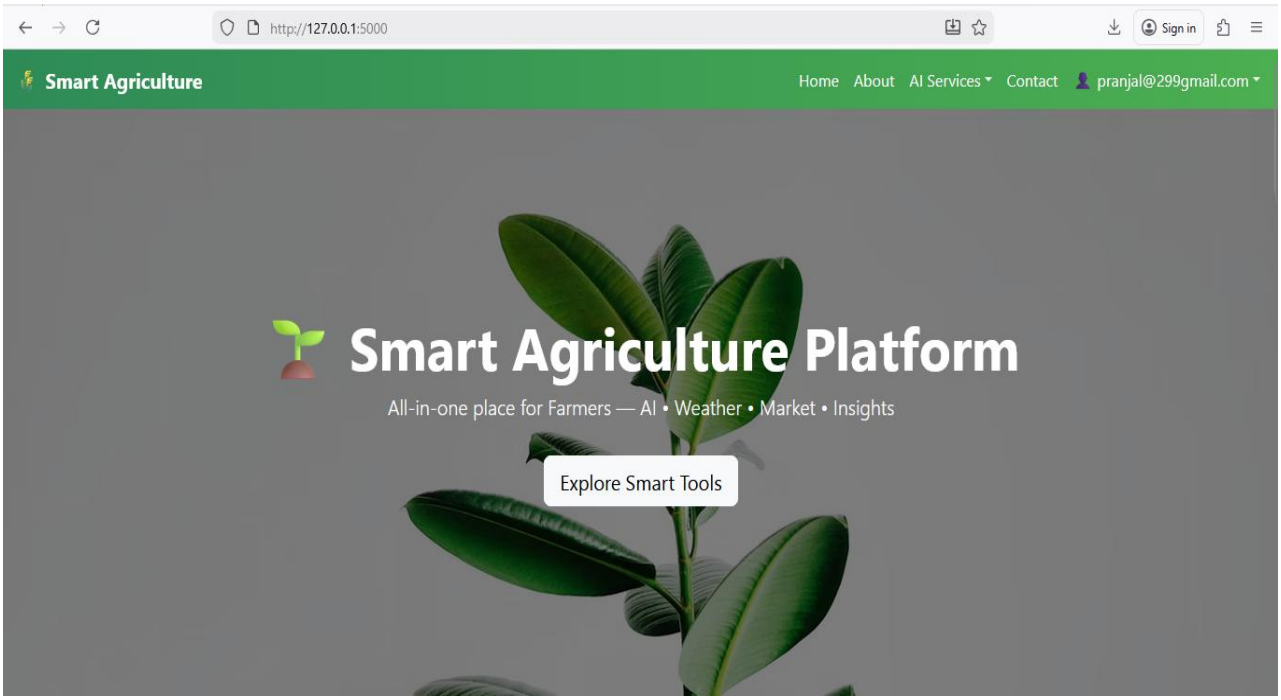


# III)Dash-board 3:

\




# I) Home Page

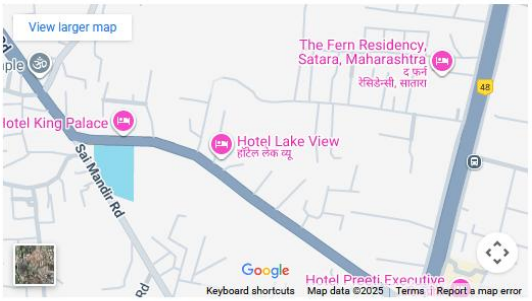


## Real-Time Farm Insights

| Temperature | Humidity | Soil Moisture |
|---|---|---|
| 26.3°C | 63% | 66% |

Live Smart Farm Monitoring

### SmartAgri Support Center




Official Help & Support Point

http://127.0.0.1:5000

## Explore Smart Tools

**Best Crop To Plant**
Based on soil & weather data
Analyze Now

**Yield Prediction**
Predict crop yield before harvest
Predict Now

**Disease Detection**
Upload leaf photo for instant diagnosis
Upload Photo

**Seed Quality Detection**
Detect seed viability using AI
Check Quality

**Rainfall Prediction**
Forecast rainfall for your region
Check Forecast

**Climate Advisory System**
AI-based farm advisory
View Advisory

**Mixed Crop Planning**
Suggests compatible crops
Plan Now

**Water Requirement Forecasting**
AI-based irrigation insights
Calculate Now

**Market Demand Forecasting**
Predict which crop will sell best
Forecast Demand

http://127.0.0.1:5000

## Live Market Prices

| Maize | Onion | Rice | Sugarcane | Tomato | Wheat |
|---|---|---|---|---|---|
| ₹23.4/kg | ₹22.8/kg | ₹54.6/kg | ₹4.2/kg | ₹34.7/kg | ₹18.1/kg |

### Smart Online Agriculture

Empowering farmers with technology, data, and innovation. We provide AI-driven insights, IoT monitoring, and sustainable farming solutions.

### Quick Links

Home
About
Contact
Login / Register

### Our Services

Crop Recommendation
Yield Prediction
Disease Detection
Market Insights

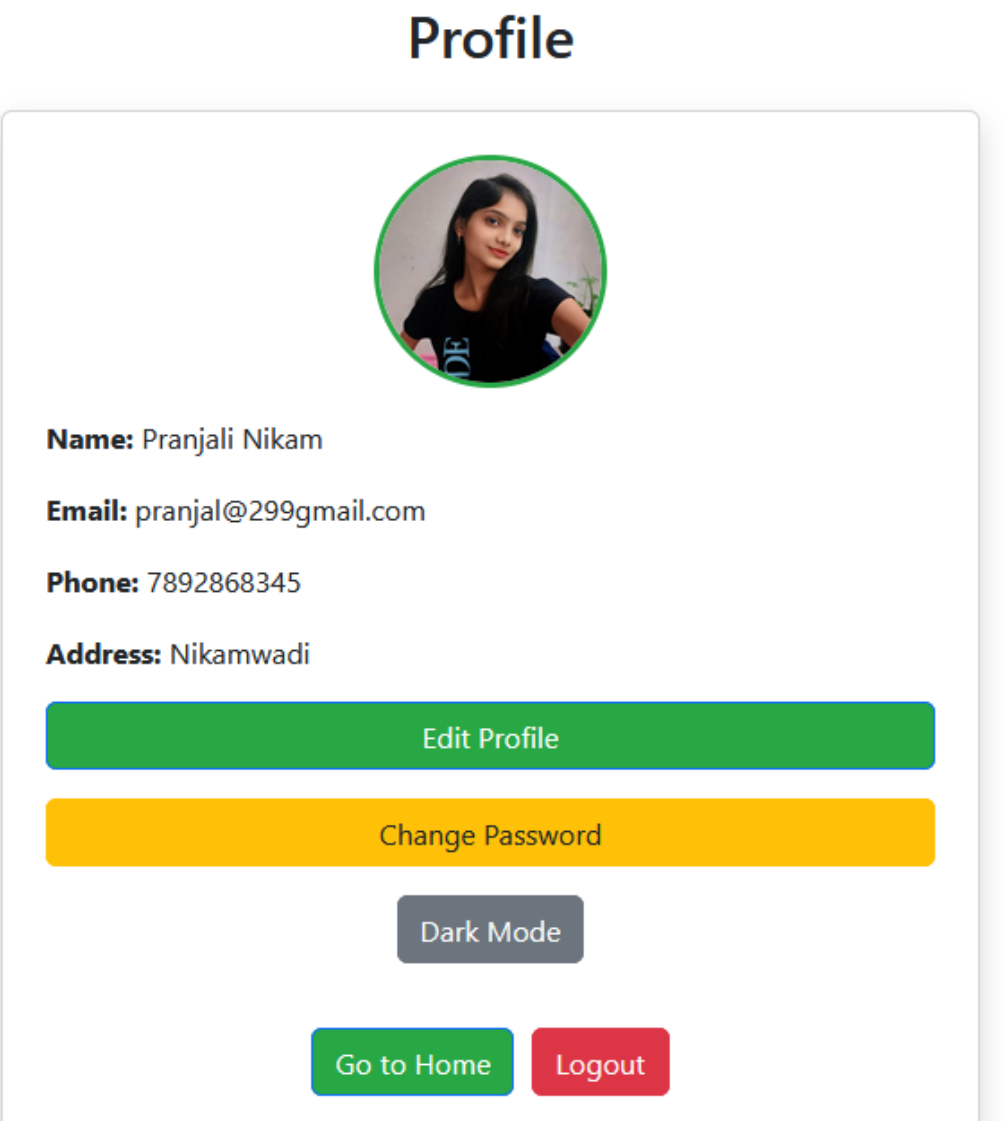
### Contact Us

123 Farm Street, City, Country
info@smartagriculture.com
+91 123 456 7890

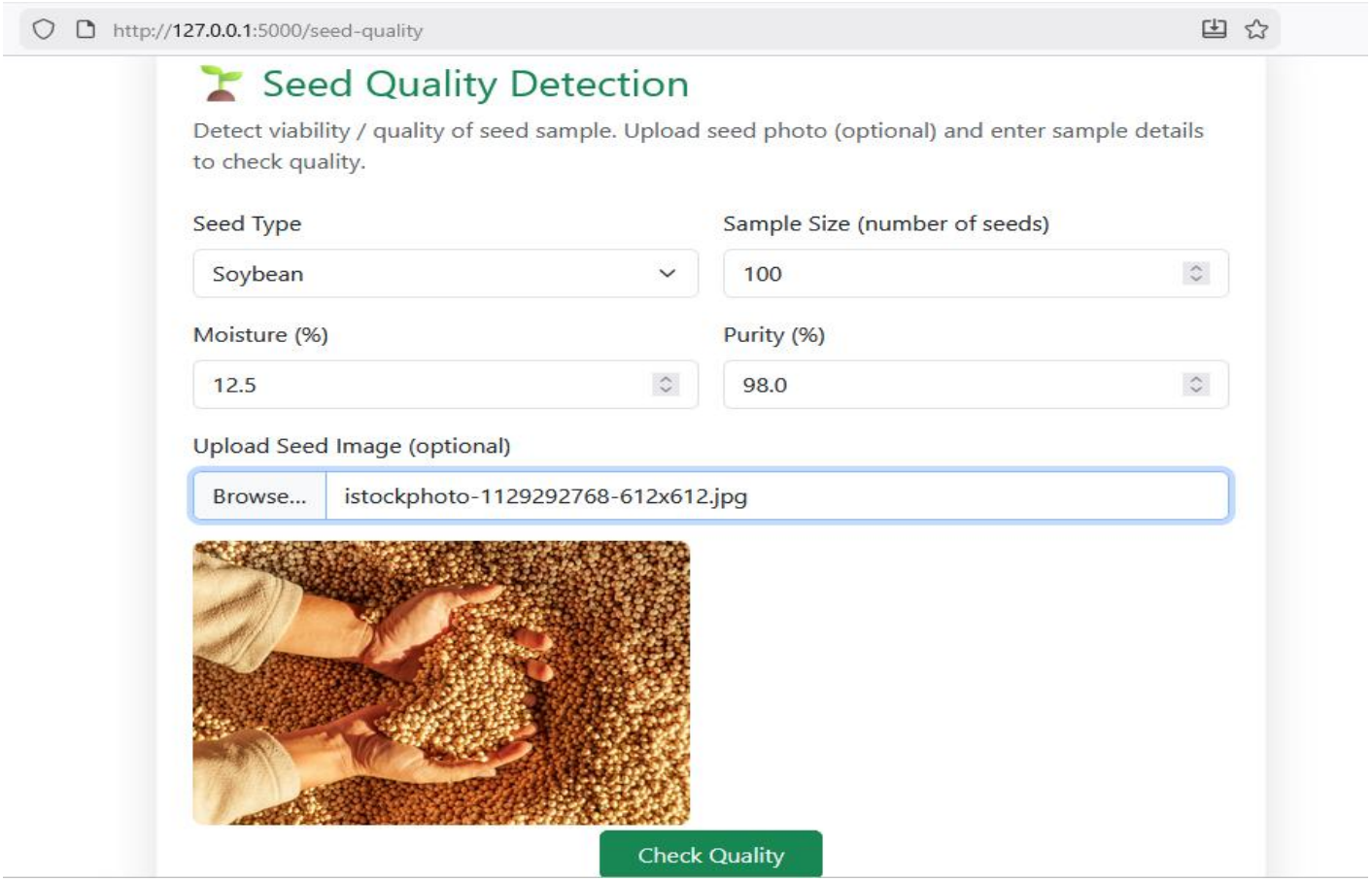
© 2025 Smart Online Agriculture Project | Designed with ♥ for Farmers

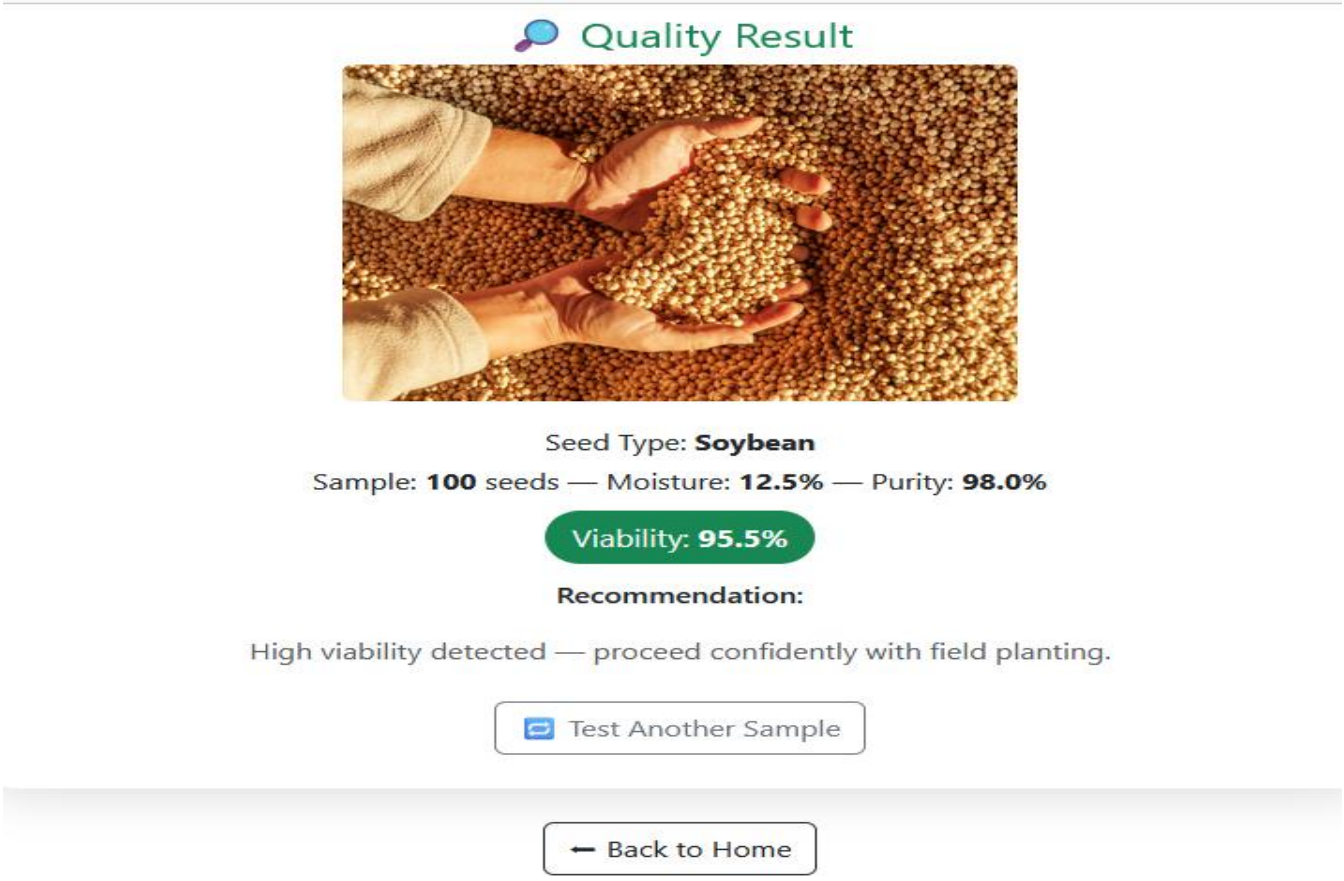
ENG IN 12:43 15-12-2025

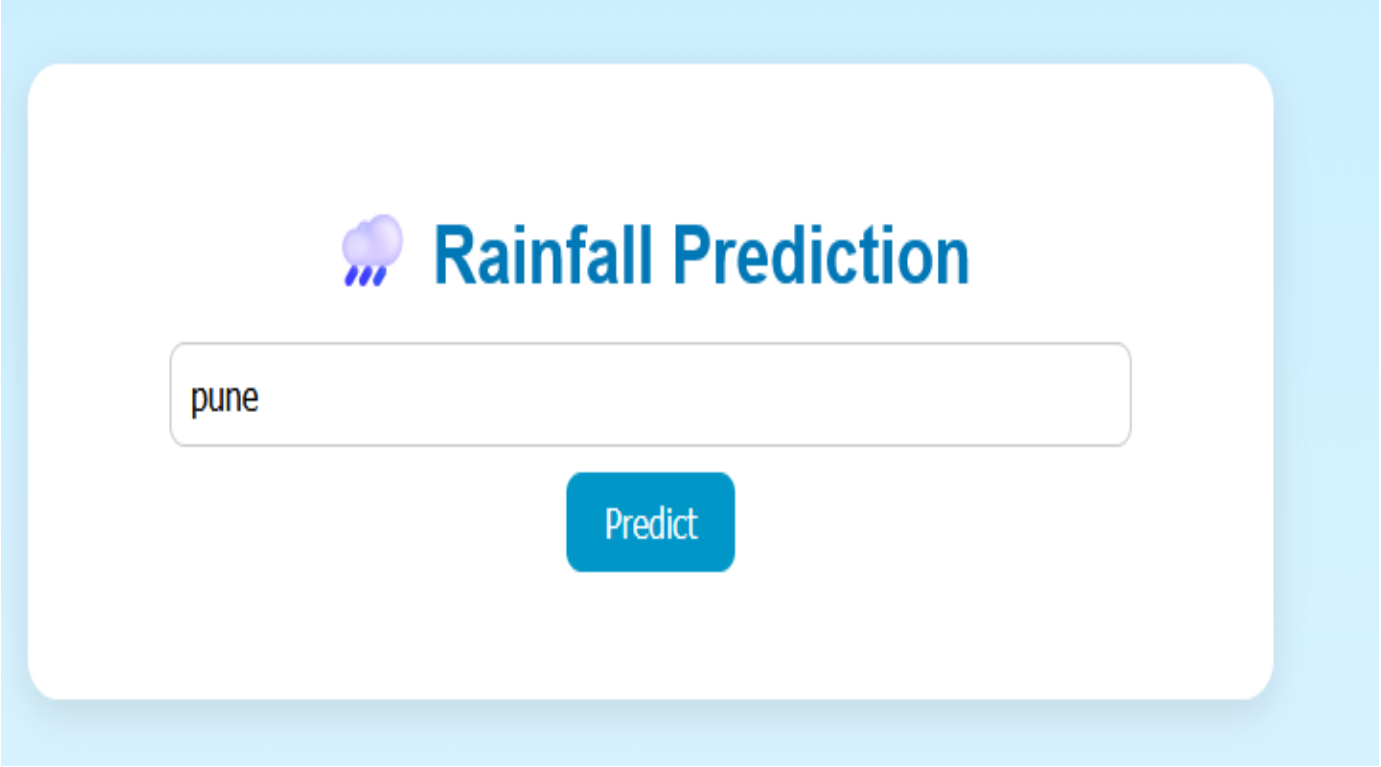
## 1.5 User Profile:

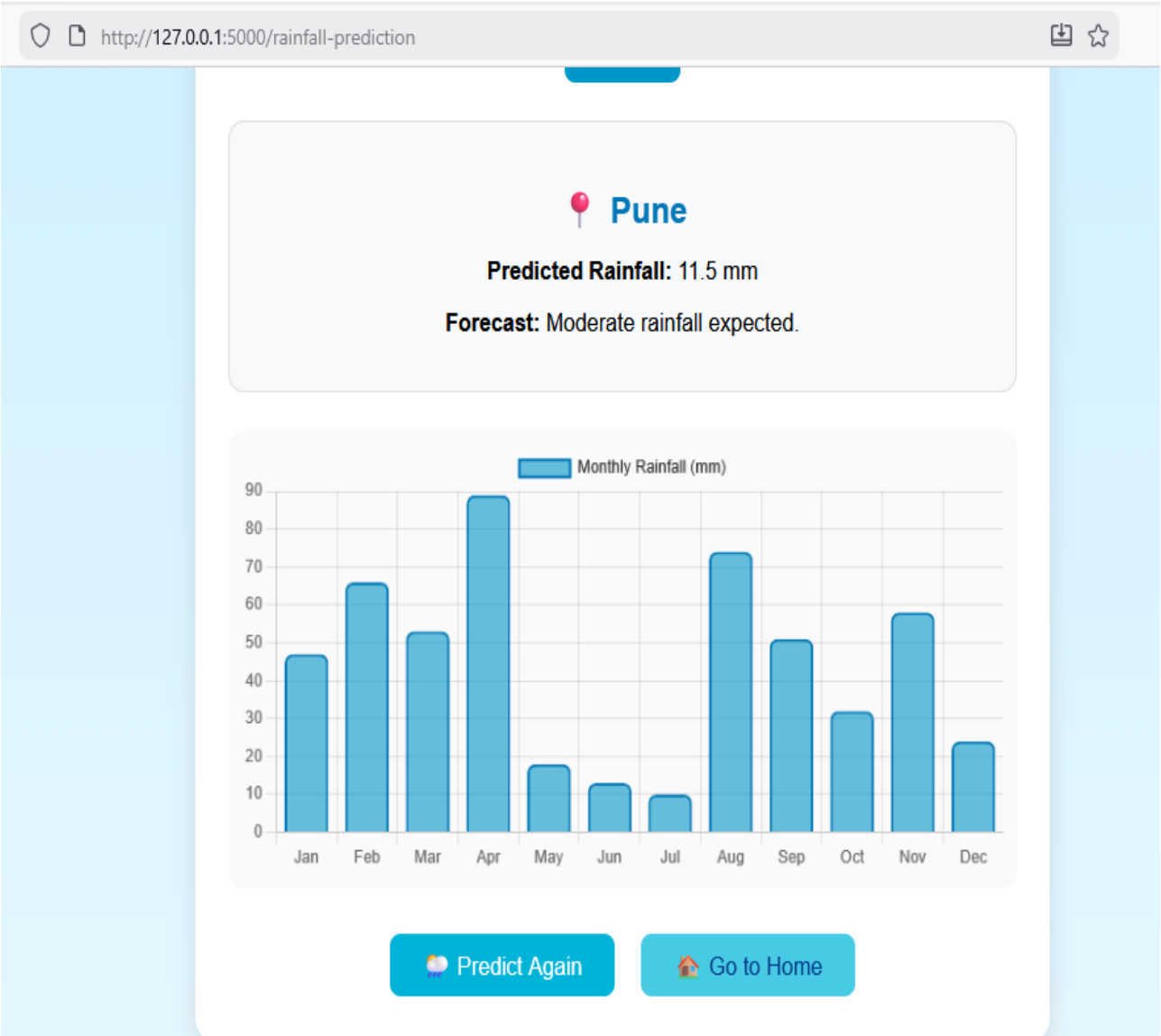


## 1.6 **Output Screens with data :**




Rainfall Prediction
pune
Predict


http://127.0.0.1:5000/rainfall-prediction
Pune
Predicted Rainfall: 11.5 mm
Forecast: Moderate rainfall expected.
Monthly Rainfall (mm)
90
80
70
60
50
40
30
20
10
0
Jan
Feb
Mar
Apr
May
Jun
Jul
Aug
Sep
Oct
Nov
Dec
Predict Again
Go to Home

## 4.3 Data Reports :

### 1.6.1 User Test Summary Report

#### User login report

| st Case | Description | Actual Result | Status |
|---|---|---|---|
| -01 | User Registration | Successful registration | ✓ Pass |
| -02 | Login (Valid credentials) | Redirected to dashboard | ✓ Pass |
| -03 | Login (Invalid password) | Error displayed | ✓ Pass |
| -04 | Forgot Password | Password reset successful | ✓ Pass |

#### Crop Data Report

| ase | Description | Expected Result | Status |
|---|---|---|---|
| 3 | Add Crop | Crop added to database | ✓ Pass |
| 9 | View Crops | Crop list displayed | ✓ Pass |
| ) | Edit Crop | Crop updated | ✓ Pass |
| | Delete Crop | Crop removed | ✓ Pass |

## 1.7 Sample program code :

### 1.6.2 app.py

```
from flask import Flask, render_template, request, session, redirect, url_for, flash, jsonify
from flask_mysqldb import MySQL
from flask_sqlalchemy import SQLAlchemy
from werkzeug.security import generate_password_hash, check_password_hash
from werkzeug.utils import secure_filename
from apscheduler.schedulers.background import BackgroundScheduler
from datetime import datetime
import os
import re
import random
import time
import threading

# ===================== Flask Setup =====================
app = Flask(__name__)
app.secret_key = "your_secret_key"

# ===================== MySQL Configuration ====================
app.config['MYSQL_HOST'] = 'localhost'
app.config['MYSQL_USER'] = 'root'
app.config['MYSQL_PASSWORD'] = 'root'
app.config['MYSQL_DB'] = 'agriculture_db'

mysql = MySQL(app)
# ===================== SQLAlchemy Configuration =====================
app.config['SQLALCHEMY_DATABASE_URI'] =
'mysql://root:root@localhost/agriculture_db'
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False

db = SQLAlchemy(app)

# ===================== User Model =====================
class User(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    username = db.Column(db.String(150), unique=True, nullable=False)
    latitude = db.Column(db.Float, nullable=True)
longitude = db.Column(db.Float, nullable=True)
# ===================== File Upload Path ================
```

```
UPLOAD_FOLDER = 'static/uploads'
os.makedirs(UPLOAD_FOLDER, exist_ok=True)
app.config['UPLOAD_FOLDER'] = UPLOAD_FOLDER

# ===================== Allowed Extensions =====================
ALLOWED_EXTENSIONS = {'png', 'jpg', 'jpeg', 'gif'}
def allowed_file(filename):
    return '.' in filename and filename.rsplit('.', 1)[1].lower() in ALLOWED_EXTENSIONS

# ===================== Auto Create Tables (crops, recommendations)
=====================
def create_support_tables():
    cur = mysql.connection.cursor()
    # crops table
    cur.execute("""
        CREATE TABLE IF NOT EXISTS crops (
            id INT AUTO_INCREMENT PRIMARY KEY,
            name VARCHAR(150),
            image VARCHAR(255),
            description TEXT,
            duration VARCHAR(100),
            fertilizer VARCHAR(255),
            expected_profit FLOAT,
            sowing_season VARCHAR(100),
            created_at DATETIME DEFAULT CURRENT_TIMESTAMP
        )
    """)
    # recommendations table
    cur.execute("""
        CREATE TABLE IF NOT EXISTS recommendations (
            id INT AUTO_INCREMENT PRIMARY KEY,
            soil_type VARCHAR(100),
            nitrogen FLOAT,
            phosphorus FLOAT,
            potassium FLOAT,
            ph FLOAT,
            temperature FLOAT,
            humidity FLOAT,
            rainfall FLOAT,
            created_at DATETIME DEFAULT CURRENT_TIMESTAMP
        )
    """)
    mysql.connection.commit()
    cur.close()
```

## 1.7 Future Scope :

- Enhanced Security – Implement JWT/OAuth authentication, two-factor authentication, password policies, and encrypted sensitive data storage.
- Role-Based Access Control – Add more user roles (farmer, expert, admin, analyst) with fine-grained permissions and customized dashboards.
- Scalability & Performance – Use caching (Redis), background task queues (Celery), and load balancing for high traffic handling.
- Modern Frontend Integration – Develop a responsive UI using React, Vue, or Angular and provide REST/GraphQL APIs for mobile apps.
- Advanced Database Management – Optimize queries, add indexing, use migrations, enable backups, and support cloud databases.
- AI & Machine Learning Features – Improve crop recommendation, disease detection, yield prediction, and market forecasting using ML models.
- Real-Time Data Integration – Integrate live weather APIs, soil sensors, satellite data, and market price feeds.
- Analytics & Reporting – Add dashboards, usage analytics, downloadable reports, and predictive insights for admins.
- Automation & Notifications – Enable email/SMS/app notifications for weather alerts, irrigation reminders, and market trends.
- Testing & DevOps – Introduce automated testing, CI/CD pipelines, Docker-based deployment, and cloud hosting.
- Localization & Accessibility – Support multiple languages, regional settings, and accessibility standards..

## 1.8 Limitations and Bibliography

### Limitations:

1. **Data Accuracy Dependency** – The system's predictions and advisories depend heavily on the accuracy of input data such as sensor readings, user inputs, and external APIs. Inaccurate data can affect results.
2. **Limited Real-Time Integration** – Currently, real-time weather, market, and satellite data integration is limited or simulated, which may reduce prediction reliability.
3. **Scalability Constraints** – Performance may degrade with a large number of users due to single-server deployment and lack of load balancing.
4. **Internet Dependency** – Most features require a stable internet connection, which can be challenging in rural or remote areas.
5. **Basic Machine Learning Models** – Some prediction modules use simplified or rule-based logic rather than advanced, continuously trained ML models.
6. **Security Limitations** – Advanced security features such as two-factor authentication and data encryption at rest are not fully implemented.
7. **Hardware Dependency** – IoT-based features depend on the availability, calibration, and maintenance of sensors.
8. **Regional Generalization** – Crop recommendations and advisories may not perfectly fit all regions due to variations in soil, climate, and farming practices.

### Bibliography


1. FAO. *Climate-Smart Agriculture Sourcebook*. Food and Agriculture Organization of the United Nations.
2. Government of India. *Agrometeorological Advisory Services (AAS)*. Ministry of Agriculture & Farmers Welfare.
3. Ian Goodfellow, Yoshua Bengio, Aaron Courville. *Deep Learning*. MIT Press, 2016.
4. Aurélien Géron. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. O'Reilly Media.
5. Flask Documentation. *Flask Web Framework*.
6. MySQL Documentation. *MySQL Database Management System*.
7. APScheduler Documentation. *Task Scheduling in Python*.
8. OpenWeather API Documentation.
9. Kaggle Datasets. *Agriculture and Crop Yield Data*.

## ➢ *Books & Research Papers :*

Ian Goodfellow, Yoshua Bengio, Aaron Courville, Deep Learning, MIT Press, 2016.

1) François Chollet, Deep Learning with Python, Manning Publications, 2018.

2) Adrian Rosebrock, Deep Learning for Computer Vision, PyImageSearch, 2019.

## ➢ **Websites & Online Resources**

1. TensorFlow Documentation – https://www.tensorflow.org
2. Keras API Reference – https://keras.io
3. OpenCV Documentation – https://docs.opencv.org
4. MediaPipe Framework (Google Developers) – https://developers.google.com/mediapipe
5. MongoDB Manual – https://www.mongodb.com/docs/
6. Flask Framework Documentation – https://flask.palletsprojects.com/

## ➢ *Technology Tools Used :*

1. Python
2. OpenCV
3. MediaPipe
4. Flask
5. HTML & CSS
6. TensorFlow
7. NumPy

## 1.10 Conclusion

This project successfully demonstrates the development of a smart agricultural web application that integrates modern technologies such as machine learning, image processing, and web development to support farmers in making informed decisions. By combining features like crop yield prediction, plant disease detection, weather and climate advisory, market forecasting, and farm data monitoring, the system provides a comprehensive digital solution for sustainable and efficient farming. The use of tools such as Python, Flask, TensorFlow, OpenCV, and database technologies ensures reliability, scalability, and ease of use. Overall, the application highlights how technology can bridge the gap between traditional farming practices and modern data-driven agriculture, contributing to improved productivity, reduced risk, and better resource management.